SmartLLM: Smart Contract Auditing using Custom Generative AI

Jun Kevin
Unversitas Pelita Harapan
Jakarta, Indonesia
0009-0004-1304-6398

Pujianto Yugopuspito
Universitas Pelita Harapan
Jakarta, Indonesia
0000-0001-6155-163X

2025 International Conference on Computer Sciences, Engineering, and Technology Innovation (ICoCSETI) | 979-8-3315-0861-6/25/$31.00 ©2025 IEEE | DOI: 10.1109/ICoCSETI63724.2025.11019687

***Abstract*—Smart contracts, integral to decentralized finance (DeFi) and blockchain ecosystems, are increasingly vulnerable to exploits due to coding errors and complex attack vectors. Traditional static analysis tools and existing vulnerability detection methods often fail to address these challenges comprehensively, resulting in high false-positive rates and an inability to detect dynamic vulnerabilities. This paper introduces SmartLLM, a novel approach leveraging fine-tuned LLaMA 3.1 models with Retrieval-Augmented Generation (RAG) to enhance the accuracy and efficiency of smart contract auditing. By integrating domain-specific knowledge from ERC standards and employing advanced techniques such as QLoRA for efficient fine-tuning, SmartLLM achieves superior performance compared to static analysis tools like Mythril and Slither and zero-shot LLM prompting (e.g., ChatGPT-3.5 and GPT-4). Experimental results demonstrate a perfect recall of 100% and an accuracy score of 70.0%, underscoring the model's robustness in identifying vulnerabilities, including re-entrancy and access control issues. This research advances smart contract security by offering a scalable and effective auditing solution, supporting the secure adoption of decentralized applications.**

***Index Terms*—smart contracts, vulnerability detection, large language models, retrieval-augmented generation, generative AI, decentralized finance, quantization low-rank adaptation.**

I. Introduction

Since the inception of Bitcoin, blockchain technology has significantly evolved, with Ethereum emerging as a pivotal development. Ethereum, a decentralized and open-source blockchain platform, enables the creation and execution of decentralized applications (DApps) and smart contracts. Ethereum Request for Comments (ERCs) have been developed to standardize smart contracts [1]. Among these, ERC-20 is notable for defining rules for fungible tokens and ensuring interoperability across various Ethereum-based projects [2]. However, smart contracts are not immune to vulnerabilities, and violations of ERC standards can lead to severe interoperability issues, security risks, and financial losses [3], [4].

The rapid growth of blockchain and decentralized finance (DeFi) has transformed the financial landscape by offering new opportunities for peer-to-peer transactions and automated services. Despite their advantages, smart contracts are susceptible to attacks and vulnerabilities, including integer overflow, re-entrancy, and access control issues, which can result in significant financial damage and erode trust in the DeFi ecosystem [5], [6]. As smart contracts become increasingly integral to DeFi applications, the need for robust security measures and innovative auditing techniques is crucial to mitigate these risks and ensure the security and reliability of decentralized financial systems [7].

Smart contract vulnerabilities often stem from coding flaws, significantly compromising the security and functionality of decentralized applications. Common issues include integer overflow, re-entrancy attacks, and inadequate access control mechanisms, all arising from code logic errors, improper use of programming constructs, or missing validation checks [6], [8]. For instance, re-entrancy vulnerabilities occur when improper handling of external function calls allows attackers to repeatedly withdraw funds before the contract updates its balance [9]. Similarly, integer overflow happens when numerical calculations exceed the storage capacity, leading to incorrect results or exploitable conditions. Access control issues pose risks where insufficient authorization checks enable unauthorized users to manipulate contract functions [5]. Given the immutable nature of blockchain, once a smart contract is deployed, its code cannot be modified, making these vulnerabilities particularly critical. This immutability means that any security flaws can be exploited without retroactive fixes, potentially leading to substantial financial losses and eroding trust in the blockchain ecosystem [6], [7].

Static code analysis is a common technique for identifying vulnerabilities in smart contracts before deployment, as it examines the code without execution to detect potential risks. However, this approach has significant limitations, often struggling to identify complex vulnerabilities that depend on dynamic execution contexts, such as interactions between multiple contracts or external data sources [5], [8]. Additionally, static analysis may generate numerous false positives, requiring extensive manual verification to filter out benign code patterns [6]. The approach also falls short when dealing with evolving coding practices and emerging vulnerabilities, as many tools rely on predefined patterns that may not account for unconventional attack vectors. Consequently, while static analysis is an important part of the auditing process, it alone is insufficient for ensuring the security of smart contracts, highlighting the need for more advanced techniques [7].

Recent research has explored the use of Large Language Models (LLMs) for smart contract auditing, leveraging their ability to understand and generate code to detect vulnerabilities that static analysis tools may overlook [10]. LLMs can analyze code contextually, consider dynamic scenarios, and recognize complex patterns. With techniques like Retrieval-Augmented Generation (RAG) and fine-tuning, LLMs can be adapted to the rapidly changing landscape of smart contract security [7]. Integrating LLMs into the auditing process not only enhances vulnerability detection but also reduces manual verification efforts by effectively filtering false positives [9].

To address the gaps in current smart contract auditing methods, this research seeks to answer the following questions:

- **RQ1**: How effective is our model in detecting specific

Authorized licensed use limited to: Nazarbayev University. Downloaded on November 03,2026 at 21:49:34 UTC from IEEE Xplore. Restrictions apply.

types of vulnerabilities in smart contracts based on accuracy, precision, recall and F1 score?
- **RQ2**: How does our model's performance compare to existing smart contract vulnerability detection tools such as Mythril or Slither and zero-shot learning LLM such as ChatGPT-3.5 or ChatGPT-4?
- **RQ3**: What are the main opportunities for improving our model, and what challenges does it face in detecting complex vulnerabilities in smart contracts?

This paper contributes to the ongoing research by introducing a novel approach that integrates RAG with the documentation of ERC standards, utilizing the latest LLaMA 3.1 8B model. The proposed method introduces a new role, the *verificator*, responsible for fact-checking vulnerabilities using RAG with ERC documentation, alongside a reasoning role. This approach aims to enhance the accuracy of smart contract vulnerability detection, offering insights into the strengths and limitations of LLM-based auditing and providing recommendations for future advancements in the field.

II. Literature Review

A. ERC-20

The ERC-20 standard defines a set of rules for fungible tokens on the Ethereum blockchain, enabling interoperability between decentralized applications (DApps) and ensuring uniformity in token behavior [11]. The standard specifies six key functions, including *transfer* for token transfers and *approve* for delegated spending, along with two events, *Transfer* and *Approval*, to log token transactions and approvals [12].

Despite its widespread adoption, ERC-20 compliance is not without challenges. Violations of the standard can lead to issues such as unhandled return values in *transfer* functions or insufficient validation of input parameters, which increase the risk of vulnerabilities [11]. These challenges underline the need for automated auditing tools to ensure compliance and security in ERC-20 implementations.

B. Static Code Analysis

Static code analysis is a prevalent method for identifying vulnerabilities in smart contracts before deployment. It analyzes code without executing it, allowing tools like Mythril and Slither to detect common issues such as reentrancy and integer overflow [12], [13].

However, static analysis has significant limitations. It struggles to detect vulnerabilities arising from dynamic execution contexts, such as those involving external contract interactions or real-time data dependencies [12]. Additionally, the reliance on predefined patterns often leads to high false positive rates, requiring extensive manual verification to filter benign code patterns. These limitations highlight the need for more advanced auditing methods, such as those leveraging generative AI.

C. Generative AI

Generative AI represents a class of artificial intelligence systems designed to generate coherent, contextually relevant outputs based on input data [14]. These models are built on large-scale neural networks, such as transformers, which excel at understanding patterns in text and generating human-like responses. The rise of Generative AI has transformed various domains, including natural language processing, code generation, and automated reasoning [15].

In the context of smart contract auditing, Generative AI models, such as GPT and LLaMA, offer a significant advantage over traditional static analysis tools [16]. By leveraging contextual understanding and pattern recognition, these models can detect vulnerabilities that are often missed by rule-based methods. For instance, Generative AI can analyze reentrancy vulnerabilities by understanding both the code and its dynamic execution context, enabling more comprehensive assessments [6].

Generative AI also introduces flexibility through techniques like fine-tuning and retrieval-augmented generation (RAG) [17], [18]. These capabilities allow models to adapt to specific domains, such as smart contract security, by incorporating domain-specific knowledge bases like ERC documentation. However, challenges remain, including the computational cost of training and inference, reliance on large annotated datasets, and the need for interpretability in critical applications.

Despite these challenges, the application of Generative AI in vulnerability detection is a promising frontier, bridging the gap between static analysis and dynamic reasoning to enhance the security of decentralized systems.

D. LLaMA 3.1

LLaMA (Large Language Model Meta AI) is a series of large language models designed to deliver state-of-the-art performance with optimized computational efficiency [19]. The LLaMA 3.1 model introduces enhancements in contextual understanding and computational efficiency, making it particularly suited for tasks requiring deep code analysis and reasoning.

In the context of smart contract auditing, LLaMA 3.1's ability to process long sequences and recognize complex patterns in code positions it as a valuable tool. By integrating this model with fine-tuning techniques and domain-specific knowledge bases, it can effectively identify vulnerabilities that static analysis tools may overlook.

E. QLoRA: Efficient Finetuning of Quantized LLMs

QLoRA (Quantized Low-Rank Adaptation) is an efficient fine-tuning technique that significantly reduces the memory and computational requirements for large language models [20]. By employing low-rank adaptation with 4-bit quantization, QLoRA enables the fine-tuning of models like LLaMA 3.1 on resource-constrained hardware without compromising performance.

This technique uses paged optimizers to handle memory spikes efficiently, as illustrated in Fig. 1, allowing for smoother training on GPUs with limited memory capacity. In this research, QLoRA is applied to fine-tune LLaMA 3.1 using domain-specific datasets, such as smart contract code and ERC documentation. This approach ensures the model is adapted efficiently for smart contract vulnerability detection while maintaining feasibility for deployment on standard hardware setups.

III. Research Methodology

A. Data Collection

The dataset for this research includes smart contract code sourced from Etherscan, a widely-used platform for Ethereum blockchain data. The dataset includes labeled examples of vulnerable and non-vulnerable contracts, which were essential

Authorized licensed use limited to: Nazarbayev University. Downloaded on November 03,2026 at 21:49:34 UTC from IEEE Xplore. Restrictions apply.

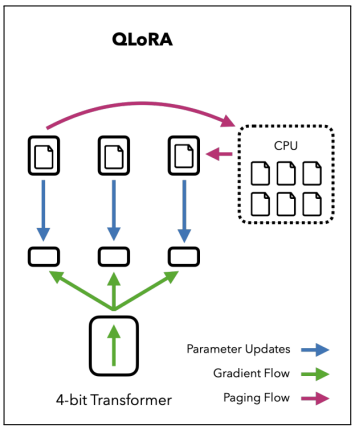


Fig. 1. How QLORA quantizing the model to 4-bit precision and using paged optimizers to handle memory spikes [20].

for training and evaluation. The dataset includes 300 smart contract samples, equally split between vulnerable and non-vulnerable contracts:

- **Vulnerable contracts:** 150 samples with three distinct vulnerability types:
  - **Re-Entrancy:** Improper handling of external calls, allowing repeated execution (**49 samples**).
  - **Access Control:** Insufficient authorization checks (**40 samples**).
  - **Logic Error:** Errors in code logic leading to unintended behaviors (**61 samples**).
- **Non-vulnerable contracts:** 150 samples with correct implementations.

The dataset was split into training and testing subsets using 80:20 ratio. Specifically, 240 contracts (120 vulnerable and 120 non-vulnerable) were used for training, while the remaining 60 contracts (30 vulnerable and 30 non-vulnerable) were used for testing.

To illustrate the vulnerabilities present in the dataset, a simplified example of a vulnerable ERC-20 smart contract is shown in Listing 1. The corpus for RAG comprises ERC-20 documentation [1], [2].

```
pragma solidity ^0.8.0;

contract ERC20 {
    string public name = "Token101";
    string public symbol = "TKN";
    uint256 public totalSupply = 1000000;

    mapping(address => uint256) public balanceOf;
    mapping(address => mapping(address => uint256)) public allowance;

    constructor() {
        balanceOf[msg.sender] = totalSupply;
    }

    function transfer(address _to, uint256 _value) public returns (bool) {
        balanceOf[msg.sender] -= _value; // No check for sufficient balance
        balanceOf[_to] += _value;
        return true;
    }

    function transferFrom(address _from, address _to, uint256 _value) public
        returns (bool) {
        balanceOf[_from] -= _value; // Missing allowance check
        balanceOf[_to] += _value;
        (bool success, ) = _to.call{value: _value}(""); // Unsafe external call
        require(success, "Transfer_failed");
        return true;
    }
}
```

Listing 1. Sample of Vurnerability Code

B. Data Preprocessing

Prior to training, the smart contract code was preprocessed to remove irrelevant comments, normalize formatting, and tokenize using the LLaMA tokenizer. The corpus for RAG was indexed using FAISS, enabling efficient retrieval of relevant context during the inference phase [21]. This preprocessing ensured consistency and optimized the model's performance.

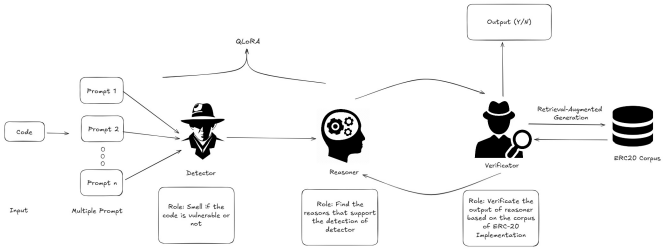


Fig. 2. Workflow diagram of SmartLLM, illustrating the roles of Detector, Reasoner, and Verificator in vulnerability detection using Retrieval-Augmented Generation.

C. Model Fine-Tuning with QLoRA

QLoRA (Quantized Low-Rank Adaptation) was employed to fine-tune the LLaMA 3.1 model on the prepared dataset. This technique reduces memory requirements by applying 4-bit quantization and low-rank adaptation, allowing efficient training on GPUs with limited resources [20]. Training parameters included a learning rate of $1e^{-4}$, a batch size of 16, and a training duration of 2 epochs.

D. Instruction Scenarios

To evaluate the model's ability to understand and detect vulnerabilities, three distinct roles were implemented within the workflow, as shown in Fig. 2:

1) **Detector:** The Detector, illustrated in Fig. 2, analyzes multiple prompts to classify smart contracts as vulnerable (`Y`) or non-vulnerable (`N`). This role ensures that potential vulnerabilities in the code are flagged for further analysis.
2) **Reasoner:** Following the Detector's output, the Reasoner identifies and provides reasons supporting the classification decision. As depicted in the workflow, this role offers detailed explanations for the detected vulnerabilities.
3) **Verificator:** The Verificator validates the outputs of the Reasoner using Retrieval-Augmented Generation (RAG) to cross-reference findings against ERC-20 corpus. This role, also illustrated in Fig. 2, ensures compliance and accuracy before generating the final classification.

These instruction scenarios were designed to comprehensively test the model's binary classification capability, its reasoning for nuanced vulnerability detection, and its ability to validate results against domain-specific knowledge.

These instruction scenarios were designed to comprehensively assess the model's binary classification capability, its reasoning for nuanced vulnerability detection, and its ability to validate results against domain-specific knowledge.

E. Evaluation Metrics

The effectiveness of the proposed approach was evaluated using the following metrics:

- **Confusion Matrix:** A table summarizing the model performance by showing the counts of true positives (TP), true negatives (TN), false positives (FP) and false

Authorized licensed use limited to: Nazarbayev University. Downloaded on November 03,2026 at 21:49:34 UTC from IEEE Xplore. Restrictions apply.

negatives (FN). This provides an overview of the model's classification capabilities.

- **Accuracy:** The proportion of correct predictions across all samples, calculated as:

$$\text{Accuracy} = \frac{\text{TP} + \text{TN}}{\text{TP} + \text{TN} + \text{FP} + \text{FN}} \tag{1}$$

- **Recall:** The model's ability to identify all vulnerable contracts, calculated as:

$$\text{Recall} = \frac{\text{TP}}{\text{TP} + \text{FN}} \tag{2}$$

- **F1 Score:** The harmonic mean of precision and recall, providing a balanced measure of the model's accuracy and completeness:

$$\text{F1 Score} = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}} \tag{3}$$

- **Precision:** The proportion of correctly predicted vulnerabilities out of all positive predictions, calculated as:

$$\text{Precision} = \frac{\text{TP}}{\text{TP} + \text{FP}} \tag{4}$$

These metrics provide a comprehensive evaluation of the model's performance, enabling comparisons with static analysis tools (e.g., Mythril, Slither) and zero-shot prompting with large language models.

IV. Results and Discussion

A. Experimental Setup

To evaluate the proposed methodology, we compared its performance against two categories of benchmarks: static code analysis tools and zero-shot prompting with large language models (LLM). For static code analysis, we selected industry-standard tools, including Mythril and Slither. For zero-shot prompting, we utilized advanced LLMs, including GPT-3.5 and GPT-4. These models were assessed without any task-specific fine-tuning, relying solely on their pre-trained knowledge to classify smart contracts as vulnerable (`Y`) or non-vulnerable (`N`). This setup tested the LLMs' ability to reason over smart contract code and provide accurate classifications directly.

Our proposed method differs significantly from these benchmarks. By integrating Retrieval-Augmented Generation (RAG), the model retrieves domain-specific knowledge from the ERC documentation during classification, enhancing its contextual understanding. Unlike static code analysis tools or zero-shot prompting, the model generates label names directly as outputs, improving interpretability and adaptability to complex scenarios.

B. Results

The evaluation of the proposed model was carried out on both training and testing datasets. Table I summarizes the performance metrics, including accuracy, recall, precision, and the F1 score. In addition, Figure 3 and Figure 4 illustrate the confusion matrices for the training and testing datasets, respectively, providing a detailed breakdown of true positives, false positives, true negatives, and false negatives.

The model demonstrated exceptional learning capabilities during training, achieving near-perfect metrics. In the test data set, the model maintained perfect recall, ensuring that all vulnerabilities were identified, with F1 score of 76.90%. The perfect recall (100%) ensures that no vulnerabilities

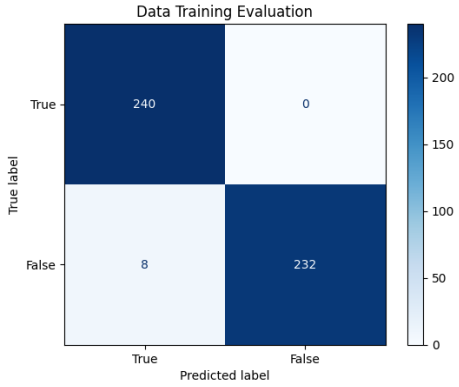


Fig. 3. Confusion Matrix for Training Dataset

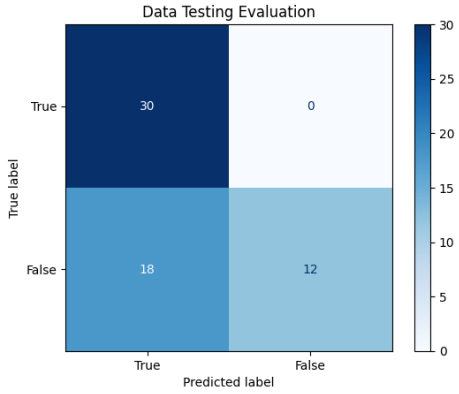


Fig. 4. Confusion Matrix for Testing Dataset

TABLE I
Evaluation Metrics for Training and Testing Data

Metric	Training Data	Testing Data
Accuracy	98.3%	70.0%
Recall	100%	100%
Precision	96.8%	62.5%
F1 Score	98.4%	76.9%

are overlooked, making the model reliable for critical applications. However, the trade-off with accuracy (70.0%) and precision (62.5%) suggests the need for further fine-tuning to reduce false positives, which is vital to minimizing manual verification efforts.

These results highlight the model's robustness in identifying specific types of vulnerabilities, such as re-entrancy and access control issues. However, the slightly lower precision in the testing phase suggests room for improvement in reducing false positives. Overall, the model is highly effective and reliable for smart contract vulnerability detection.

Table II presents a comprehensive comparison of vulnerability detection methods, including static analysis tools (Mythril and Slither), zero-shot prompting with large language models (ChatGPT-3.5 and ChatGPT-4), and the proposed approach (SmartLLM).

The static analysis tools, Mythril and Slither, were run locally on a machine with 32GB of RAM and an Intel Core Ultra 7 processor, ensuring consistent performance for comparison. For SmartLLM, the model was fine-tuned and evaluated using Google Colab Pro with NVIDIA A100 GPU and 40GB of VRAM, leveraging the platform's high computational capacity for efficient processing of large datasets.

Authorized licensed use limited to: Nazarbayev University. Downloaded on November 03,2026 at 21:49:34 UTC from IEEE Xplore. Restrictions apply.

TABLE II
COMPARISON OF VULNERABILITY DETECTION METHODS

Method	Accuracy	Recall	Precision	F1 Score
Mythril	21.7%	34.8%	20.0%	25.4%
Slither	35%	42.3%	31.4%	36.1%
ChatGPT-3.5	39.1%	38.7%	37.5%	38.1%
ChatGPT-4	43.3%	36.7%	42.3%	39.3%
SmartLLM	**70.0%**	**100%**	**62.5%**	**76.9%**

Traditional static analysis tools, Mythril and Slither, show limited effectiveness in identifying vulnerabilities. Mythril achieved an accuracy of 21.7%, with recall and precision scores of 34.8% and 20%, respectively. These results indicate that Mythril struggles to balance false positives and false negatives, resulting F1 score of 25.4%. Slither outperformed Mythril slightly, achieving an accuracy of 35%, a recall of 42.3%, and a precision of 31.4%. However, the F1 score of 36.1% suggests room for improvement, particularly in precision.

Zero-shot prompting with LLMs (ChatGPT-3.5 and ChatGPT-4) provided moderate performance. ChatGPT-3.5 achieved an accuracy of 39.1%, recall of 38.7%, precision of 37.5%, and F1 score of 38.1%. ChatGPT-4 demonstrated slightly better precision at 42.3% and F1 score of 39.3%, despite having lower recall at 36.7%. These results highlight the potential of LLMs in vulnerability detection tasks but also underline the limitations of zero-shot prompting in complex scenarios, such as handling re-entrancy vulnerabilities or access control issues.

The proposed approach, SmartLLM, significantly outperformed all baseline methods. It achieved the highest accuracy (70.0%), precision (62.5%), and F1 score (76.9%), with a perfect recall of 100%. These results indicate that the model excels at identifying all true vulnerabilities while maintaining a balanced performance across metrics. The high recall demonstrates the reliability of the model in detecting various types of vulnerabilities, including re-entrancy and logic errors, while the improved precision reflects its ability to minimize false positives compared to traditional tools and zero-shot LLMs.

Overall, the findings emphasize the effectiveness of the SmartLLM approach, particularly in scenarios where detecting critical vulnerabilities is paramount. While static tools and zero-shot prompting provide baseline capabilities, the integration of domain-specific knowledge and fine-tuning in SmartLLM delivers superior results, making it a robust solution for smart contract vulnerability detection.

C. Discussion

Table II highlights the strengths and limitations of the proposed SmartLLM approach in detecting vulnerabilities. While achieving outstanding recall (100%), a strong F1 score (76.9%) and accuracy (70.0%), there are key opportunities for improvement and challenges to address.

Opportunities: Improving precision (currently 62.5%) is essential to reduce false positives, which can be achieved through fine-tuning with more diverse and balanced datasets. Additionally, integrating dynamic execution contexts, expanding datasets to cover emerging vulnerabilities, and leveraging multi-modal inputs (e.g., audit reports) present significant opportunities to enhance the model's capability.

Challenges: The model faces token limitations that hinder processing long contracts, difficulties in generalizing across platforms, and interpreting complex attack vectors like multi-step vulnerabilities. Furthermore, the computational cost of fine-tuning and deploying large models remains a practical challenge for real-time applications.

V. CONCLUSION

This study introduced **SmartLLM**, a novel approach leveraging **Retrieval-Augmented Generation (RAG)** and fine-tuned **LLaMA 3.1** models for smart contract vulnerability detection, with a focus on improving accuracy, recall, precision, and F1 scores. By integrating ERC documentation into the vulnerability detection process, the model addresses gaps in static analysis tools and zero-shot large language models (LLMs), delivering superior results.

The proposed model achieved significant advancements over traditional tools like Mythril and Slither, and over zero-shot LLM prompting (e.g., ChatGPT-3.5 and ChatGPT-4). SmartLLM demonstrated a high recall of 100%, ensuring comprehensive detection of vulnerabilities while achieving a balanced precision of 62.5% and a strong F1 score of 76.9% and accuracy 70.0%. These results indicate the model's robustness in detecting common and complex vulnerabilities, such as re-entrancy and access control issues, surpassing conventional methods.

Key findings highlight the effectiveness of domain-specific knowledge integration and fine-tuning techniques like QLoRA in enhancing detection accuracy. The proposed system successfully bridges the gap between static analysis tools and dynamic reasoning capabilities, providing a reliable and scalable solution for auditing Ethereum smart contracts.

Despite these achievements, challenges remain, including improving the precision to reduce false positives and addressing computational resource constraints during model fine-tuning and deployment. Future work should focus on incorporating dynamic execution contexts, broader datasets covering emerging attack vectors, and larger pre-trained models (e.g., LLaMA 70B and 405B) to enhance both precision and scalability.

In summary, **SmartLLM** represents a significant step forward in ensuring the security and reliability of smart contracts, contributing to the advancement of auditing techniques in decentralized finance (DeFi) and blockchain ecosystems. By enhancing vulnerability detection and reducing manual verification efforts, the proposed approach supports the broader adoption of secure and trustworthy decentralized applications.

ACKNOWLEDGMENT

This research is partially funded by Center of Research and Community Development (LPPM), Universitas Pelita Harapan, **No. 155/LPPM-UPH/VII/2024** and **No. 156/LPPM-UPH/VII/2024**, as registered grants of the Faculty of Information Technology **No. P-113-FIT/VII/2024** and **P-114-FIT/VII/2024**, in July 2024.

REFERENCES

[1] Ethereum, *Token Standards*. Available: https://ethereum.org/en/developers/docs/standards/tokens/

[2] F. Vogelsteller and V. Buterin, *ERC-20: Token Standard*. Available: https://eips.ethereum.org/EIPS/eip-20

Authorized licensed use limited to: Nazarbayev University. Downloaded on November 03,2026 at 21:49:34 UTC from IEEE Xplore. Restrictions apply.

[3] L. Zhou *et al.*, "SoK: Decentralized Finance (DeFi) Attacks," in *2023 IEEE Symposium on Security and Privacy (SP)*, 2023, pp. 2444–2461. doi: 10.1109/SP46215.2023.10179435.
[4] Z. Zhang, B. Zhang, W. Xu, and Z. Lin, "Demystifying Exploitable Bugs in Smart Contracts," in *Proceedings of the 45th International Conference on Software Engineering*, in ICSE '23. IEEE Press, 2023, pp. 615–627. doi: 10.1109/ICSE48619.2023.00061.
[5] C. Sendner, L. Petzi, J. Stang, and A. Dmitrienko, "Vulnerability Scanners for Ethereum Smart Contracts: A Large-Scale Study," Dec. 2023.
[6] C. Chen *et al.*, "When ChatGPT Meets Smart Contract Vulnerability Detection: How Far Are We?," Sep. 2023.
[7] T. Li and L. Hu, "Audit as You Go: A Smart Contract-Based Outsourced Data Integrity Auditing Scheme for Multiauditor Scenarios with One Person, One Vote," *Security and Communication Networks*, vol. 2022, pp. 1–13, Mar. 2022, doi: 10.1155/2022/8783952.
[8] A. Ghaleb, J. Rubin, and K. Pattabiraman, "AChecker: Statically Detecting Smart Contract Access Control Vulnerabilities," in *2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE)*, IEEE, May 2023, pp. 945–956. doi: 10.1109/ICSE48619.2023.00087.
[9] D. He, Z. Deng, Y. Zhang, S. Chan, Y. Cheng, and N. Guizani, "Smart Contract Vulnerability Analysis and Security Audit," *IEEE Netw*, vol. 34, no. 5, pp. 276–282, Sep. 2020, doi: 10.1109/MNET.001.1900656.
[10] J. Yu, "Retrieval Augmented Generation Integrated Large Language Models in Smart Contract Vulnerability Detection," Jul. 2024.
[11] K. H. Ansari and U. Kulkarni, "Implementation of Ethereum Request for Comment (ERC20) Token," *SSRN Electronic Journal*, 2020, doi: 10.2139/ssrn.3561395.
[12] T. Durieux, J. F. Ferreira, R. Abreu, and P. Cruz, "Empirical review of automated analysis tools on 47,587 Ethereum smart contracts," in *Proceedings of the ACM/IEEE 42nd International Conference on Software Engineering (ICSE '20)*, New York, NY, USA: Association for Computing Machinery, 2020, pp. 530–541, doi: 10.1145/3377811.3380364.
[13] J. Feist, G. Grieco, and A. Groce, "Slither: A Static Analysis Framework for Smart Contracts," in *2019 IEEE/ACM 2nd International Workshop on Emerging Trends in Software Engineering for Blockchain (WETSEB)*, 2019, pp. 8–15, doi: 10.1109/WETSEB.2019.00008.
[14] S. Feuerriegel, J. Hartmann, C. Janiesch, and P. Zschech, "Generative AI," *Business & Information Systems Engineering*, vol. 66, no. 1, pp. 111–126, Feb. 2024, doi: 10.1007/s12599-023-00834-7.
[15] E. Brynjolfsson, D. Li, and L. R. Raymond, "Generative AI at Work," Apr. 2023, doi: 10.3386/w31161.
[16] S. Hu, T. Huang, F. İlhan, F. Tekin, and L. Liu, "Large Language Model-Powered Smart Contract Vulnerability Detection: New Perspectives," [Online]. Available: https://github.com/git-disl/GPTLens.
[17] N. Ding et al., "Parameter-efficient fine-tuning of large-scale pre-trained language models," *Nature Machine Intelligence*, vol. 5, no. 3, pp. 220–235, 2023, doi: 10.1038/s42256-023-00626-4.
[18] P. Lewis et al., "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks," in *Advances in Neural Information Processing Systems*, H. Larochelle, M. Ranzato, R. Hadsell, M. F. Balcan, and H. Lin, Eds., Curran Associates, Inc., 2020, pp. 9459–9474. https://proceedings.neurips.cc/paper_files/paper/2020/file/6b493230205f780e1bc26945df7481e5-Paper.pdf.
[19] H. Touvron et al., "LLaMA: Open and Efficient Foundation Language Models," 2023. https://arxiv.org/abs/2302.13971.
[20] T. Dettmers, A. Pagnoni, A. Holtzman, and L. Zettlemoyer, "QLoRA: Efficient Finetuning of Quantized LLMs," in *Advances in Neural Information Processing Systems*, A. Oh, T. Naumann, A. Globerson, K. Saenko, M. Hardt, and S. Levine, Eds., Curran Associates, Inc., 2023, pp. 10088–10115. https://proceedings.neurips.cc/paper_files/paper/2023/file/1feb87871436031bdc0f2beaa62a049b-Paper-Conference.pdf.
[21] M. Douze et al., "The Faiss library," 2024. [Online]. Available: https://arxiv.org/abs/2401.08281.

Authorized licensed use limited to: Nazarbayev University. Downloaded on November 03,2026 at 21:49:34 UTC from IEEE Xplore. Restrictions apply.